

Trabajo Integrador de Aplicaciones Interactivas

El Trabajo Práctico Obligatorio (TPO) consiste en el desarrollo progresivo de una **aplicación web transaccional con funcionalidades de e-commerce**, implementada mediante una arquitectura cliente-servidor.

El proyecto se desarrollará en **dos etapas obligatorias**, correspondientes a dos entregas incrementales. Cada etapa deberá constituir un sistema funcional y verificable, y deberá servir como base para la siguiente.

El trabajo podrá realizarse individualmente o en grupos de **2 a 4 integrantes**.

La evaluación será **individual**, independientemente de la modalidad grupal. Cada integrante deberá poder explicar y justificar la arquitectura, tecnologías, patrones de diseño, estructuras de datos, decisiones de implementación y funcionamiento de las partes desarrolladas.

ETAPA 1 — BACK-END Y PERSISTENCIA

Objetivo:

Desarrollar el **back-end de la aplicación**, implementando la lógica de negocio, persistencia de datos y una interfaz de comunicación mediante red que permita al futuro front-end interactuar con el sistema.

La aplicación deberá ejecutarse inicialmente en un entorno local mediante **localhost**, pero deberá estar diseñada bajo una arquitectura **cliente-servidor**, evitando soluciones que dependan exclusivamente de una ejecución local o de una comunicación directa entre la interfaz y la base de datos.

Tecnologías:

- **Java.**
- **Framework establecido por la cátedra.** (Spring)
- **SQL** y un sistema gestor de base de datos.
- Comunicación mediante **HTTP**.
- API de comunicación entre cliente y servidor.
- Formato de intercambio de datos adecuado.

Durante la entrega deberá demostrarse el funcionamiento del back-end mediante herramientas que permitan realizar solicitudes HTTP.

Debe estar acompañado por una documentación que explique el proceso de desarrollo del ecosistema planteado. Con detalles exhaustivos y bien definidos sobre que hace cada cosa en su sistema.

- Identificación de capas en la estructura de software
- Conocimiento de las responsabilidades de las clases planteadas.
- Explicación sobre la sintaxis de los elementos requeridos.
- Justificación sobre las decisiones de diseño aplicadas.

Requerimientos:

El sistema deberá implementar, como mínimo:

1. **Arquitectura organizada por capas o responsabilidades.**
2. Conexión y persistencia mediante una base de datos SQL.
3. Modelado de las entidades principales del dominio.
4. Operaciones **CRUD** sobre las entidades correspondientes.
5. Implementación de la lógica de negocio.
6. Exposición de endpoints para permitir la comunicación con clientes externos.
7. Manejo adecuado de solicitudes HTTP y respuestas.

8. Validación de datos de entrada.
9. Manejo de errores y respuestas apropiadas.
10. Configuración que permita ejecutar el servidor mediante localhost.
11. Documentación básica de los endpoints disponibles.
12. Separación entre la lógica de negocio, acceso a datos y capa de comunicación.
13. El sistema deberá poder ejecutarse y responder solicitudes en tiempo real.

Resultado esperado:

Se debe contar con un servidor capaz de recibir solicitudes externas, procesarlas, ejecutar la lógica correspondiente y consultar/modificar información persistida en la base de datos.

El front-end definitivo **no forma parte de esta entrega**.

ETAPA 2 — INTEGRACIÓN FRONT-END + BACK-END

Objetivo:

Desarrollar e integrar la **interfaz de usuario**, utilizando el back-end desarrollado durante la primera etapa como servidor de la aplicación.

El objetivo de esta etapa es transformar el back-end previamente desarrollado en una **aplicación web funcional e interactiva**, accesible mediante una interfaz gráfica.

Tecnologías mínimas

El front-end deberá utilizar:

- **HTML5.**
- **CSS3.**
- **JavaScript.**
- Comunicación con el back-end mediante **HTTP**.
- Intercambio de información mediante el formato definido para la API.

La aplicación deberá:

1. Implementar una interfaz web funcional.
2. Consumir los endpoints desarrollados durante la primera etapa.
3. Mostrar información obtenida desde la base de datos.
4. Permitir al usuario realizar operaciones sobre el sistema.
5. Implementar validaciones en la interfaz.
6. Gestionar adecuadamente estados de carga y errores.
7. Mantener una separación clara entre presentación y lógica del servidor.
8. Utilizar la API del back-end para las operaciones de datos.
9. Implementar las funcionalidades principales definidas para el dominio seleccionado.
10. Ser demostrable en tiempo real.

Condición fundamental del TPO

La aplicación **no deberá quedar planteada como una solución exclusivamente local o monolítica**.

Aunque durante las primeras etapas pueda ejecutarse mediante localhost, la arquitectura deberá permitir que el cliente y el servidor se comuniquen mediante protocolos de red y que, mediante la configuración correspondiente, el servidor pueda ser trasladado a otro equipo o entorno de ejecución.

La comunicación deberá respetar una arquitectura cliente-servidor: Front-end → HTTP → Back-end → Base de datos.

Evaluación

La evaluación contemplará tanto el **resultado funcional** como la comprensión individual del desarrollo.

Cada integrante podrá ser evaluado sobre:

- arquitectura;
- patrones de diseño;
- principios de desarrollo de software;
- estructura del proyecto;
- funcionamiento del framework;
- comunicación HTTP;
- diseño y consumo de API;
- persistencia SQL;
- lógica de negocio;
- integración front-end/back-end;
- decisiones técnicas adoptadas;
- capacidad para modificar, extender o justificar el código desarrollado.

La pertenencia a un grupo no implica que todos los integrantes reciban automáticamente la misma calificación.

Preguntas de guía para ir completando a medida que avanza el proyecto:

Arquitectura

- ¿Por qué dividieron el proyecto en estas capas?
- ¿Qué responsabilidad tiene cada capa?
- ¿Qué ocurriría si colocáramos toda la lógica en el Controller?
- ¿Dónde debería estar una regla de negocio y por qué?
- ¿Qué componente debería modificarse si cambiamos el motor de base de datos?

MVC

- ¿Dónde está el Model en su aplicación?
- ¿Cuál es la responsabilidad del Controller?
- ¿Qué diferencia hay entre Controller y Service?
- ¿Por qué la View no debería acceder directamente a la base de datos?

DAO

- ¿Qué problema resuelve DAO?
- ¿Qué responsabilidad tiene?
- ¿Qué ocurriría si elimináramos esta capa?
- ¿DAO y ORM son lo mismo?

ORM

- ¿Qué significa Object-Relational Mapping?
- ¿Qué relación existe entre una Entity y una tabla?
- ¿Qué representa una relación @OneToMany?
- ¿Qué ventajas tiene utilizar ORM?
- ¿Qué problemas puede generar una mala utilización del ORM?

SQL

- ¿Por qué esta tabla tiene esa clave primaria?
- ¿Qué relación existe entre estas tablas?
- ¿Qué sucede cuando eliminás este registro?

- ¿Dónde se está realizando realmente la persistencia?

HTTP/API

- ¿Qué diferencia hay entre GET y POST?
- ¿Por qué utilizaron este endpoint?
- ¿Qué significa un HTTP 404?
- ¿Qué información viaja en el request?
- ¿Qué devuelve el servidor?
- ¿Qué formato de archivo usamos y por qué no otro?

Diseño

- ¿Qué patrón están utilizando?
- ¿Por qué eligieron ese patrón?
- ¿Qué problema solucionaría?
- ¿Podrían haberlo hecho de otra manera?
- ¿Qué ventaja aporta esta decisión?

Rubrica de evaluación del proyecto:

Rúbrica de evaluación del Trabajo Integrador de Aplicaciones Interactivas

La evaluación del Trabajo Integrador contemplará tanto el **funcionamiento técnico de la aplicación** como la **comprensión individual de las decisiones y tecnologías utilizadas**.

La calificación podrá considerar diferencias entre integrantes de un mismo grupo cuando se evidencie una participación, comprensión o dominio técnico diferente.

Criterios de evaluación

Criterio	Descripción	Ponderación
Arquitectura y organización	Separación adecuada de responsabilidades, organización por capas, estructura del proyecto y aplicación de principios de diseño.	10%
Persistencia y SQL	Modelado de la base de datos, relaciones, claves, consultas, operaciones CRUD y correcta persistencia de la información.	10%
Modelado del dominio y ORM	Correcta definición de entidades, relaciones entre entidades y utilización del ORM/framework establecido por la cátedra.	10%
Lógica de negocio	Implementación de reglas de negocio en la capa correspondiente, evitando trasladar responsabilidades al Controller o a otras capas incorrectas.	10%
API y comunicación HTTP	Diseño y funcionamiento de endpoints, utilización correcta de métodos HTTP, parámetros, request, response y códigos de estado.	10%
Validaciones y manejo de errores	Validación de datos de entrada, tratamiento de errores y generación de respuestas adecuadas para el cliente.	10%
Funcionamiento e integración	Capacidad del sistema para ejecutarse en tiempo real, comunicarse mediante HTTP y, en la Etapa 2, integrar correctamente el front-end con el back-end.	15%
Documentación y justificación técnica	Documentación de endpoints, arquitectura, decisiones de diseño, tecnologías utilizadas y explicación del funcionamiento del sistema.	5%
Comprensión individual	Capacidad de explicar, justificar, modificar y extender el código desarrollado, respondiendo preguntas técnicas sobre el proyecto.	20%

Si alguno de los criterios es calificado con 0, el trabajo queda automáticamente desaprobado. Siendo este el trabajo principal de la materia, esto significa que recursa la materia en caso de no tener el trabajo realizado o no saber nada sobre la estructura y funcionamiento del mismo. Se considera también la entrega en tiempo y forma, siendo que si hay atrasos con la misma se descuenta del puntaje total 3 puntos, siendo así que la máxima nota será 7.

Evaluación de la Etapa 1

La primera etapa se evaluará considerando los siguientes aspectos:

Arquitectura

- Existencia de una arquitectura organizada por capas o responsabilidades.
- Separación entre Controller, Service y acceso a datos.
- Correcta distribución de responsabilidades.
- Organización coherente del proyecto.
- Utilización justificada de patrones de diseño cuando corresponda.

Persistencia

- Conexión funcional con una base de datos SQL.
- Correcto modelado de tablas y relaciones.
- Utilización adecuada de claves primarias y foráneas.
- Operaciones de creación, consulta, modificación y eliminación.
- Persistencia correcta de los datos.

Entidades y ORM

- Correcto modelado de las entidades del dominio.
- Correspondencia entre entidades y estructuras persistentes.
- Correcta utilización de anotaciones y configuraciones del ORM.
- Implementación adecuada de relaciones entre entidades.

Lógica de negocio

- Implementación de las reglas de negocio en la capa correspondiente.
- Separación entre reglas de negocio y comunicación HTTP.
- Evitar concentrar toda la lógica dentro de los Controllers.
- Validación de condiciones necesarias para realizar las operaciones.

API REST y HTTP

- Definición clara de endpoints.
- Utilización correcta de métodos HTTP.
- Manejo de parámetros y datos enviados en las solicitudes.
- Generación de respuestas apropiadas.
- Utilización adecuada de códigos de estado HTTP.
- Intercambio de información mediante el formato definido para la API.

Validaciones y errores

- Validación de los datos recibidos.
- Identificación de solicitudes incorrectas.
- Manejo de excepciones.
- Respuestas comprensibles para el cliente.
- Utilización adecuada de códigos HTTP de error.

Pruebas

El funcionamiento del back-end deberá demostrarse mediante una herramienta capaz de realizar solicitudes HTTP.

Se deberá poder demostrar, como mínimo:

- Consultas mediante **GET**.
- Creación de recursos mediante **POST**.
- Modificación mediante **PUT/PATCH**, según el diseño adoptado.
- Eliminación mediante **DELETE**.
- Casos exitosos.
- Casos con datos inválidos o recursos inexistentes.

Evaluación de la Etapa 2

La segunda etapa evaluará la integración del sistema completo.

Front-end

- Utilización de HTML5, CSS3 y JavaScript.
- Interfaz funcional y comprensible.
- Organización adecuada de los componentes de la interfaz.
- Validación de datos desde el cliente.
- Manejo de estados de carga.
- Manejo de errores.

Integración

- Comunicación entre front-end y back-end mediante HTTP.
- Consumo correcto de los endpoints desarrollados durante la Etapa 1.
- Visualización de información obtenida desde la base de datos.
- Envío de información desde la interfaz hacia el servidor.
- Actualización de la interfaz luego de las operaciones realizadas.

Arquitectura completa

El sistema deberá respetar el flujo:

Front-end → HTTP → Back-end → Lógica de negocio → Persistencia → Base de datos

El front-end no deberá acceder directamente a la base de datos.

Evaluación individual

La evaluación individual tendrá especial importancia debido a que el trabajo puede realizarse en grupos.

Durante la instancia de evaluación, cada integrante podrá ser solicitado a:

- Explicar la arquitectura utilizada.
- Identificar las diferentes capas del sistema.
- Explicar la responsabilidad de sus clases.
- Explicar el funcionamiento de los Controllers, Services y componentes de persistencia.
- Explicar el funcionamiento del ORM utilizado.
- Explicar las relaciones entre las entidades.
- Explicar las operaciones SQL involucradas.
- Explicar el funcionamiento de los endpoints.
- Interpretar una solicitud HTTP.
- Interpretar una respuesta HTTP.
- Explicar los códigos de estado utilizados.
- Justificar decisiones de diseño.
- Modificar una funcionalidad existente.
- Agregar una funcionalidad sencilla.
- Corregir un error introducido deliberadamente.
- Explicar qué ocurriría si se modificara una parte determinada de la arquitectura.

La capacidad de **explicar y modificar el código** será considerada evidencia de comprensión del desarrollo realizado.

Niveles de desempeño

Nivel excelente

El estudiante puede explicar el funcionamiento completo de la solución, justificar las decisiones técnicas adoptadas y realizar modificaciones o extensiones sobre el código sin depender exclusivamente de una guía externa.

Nivel satisfactorio

El estudiante comprende la arquitectura general, puede explicar las principales tecnologías utilizadas y puede realizar modificaciones sobre las partes fundamentales del sistema con una asistencia mínima.

Nivel básico

El sistema presenta un funcionamiento parcial o correcto, pero el estudiante presenta dificultades para explicar las decisiones adoptadas, las responsabilidades de las clases o el funcionamiento de determinadas tecnologías.

Nivel insuficiente

El sistema no cumple los requerimientos fundamentales o el estudiante no puede explicar de manera suficiente el funcionamiento del código que forma parte de su entrega.

Condiciones para la aprobación

Para aprobar el Trabajo Integrador se deberá cumplir con los requerimientos funcionales y técnicos establecidos para la etapa correspondiente.

Además:

- El sistema deberá poder ejecutarse y demostrarse en tiempo real.
- La comunicación entre cliente y servidor deberá realizarse mediante HTTP.
- La persistencia deberá realizarse mediante una base de datos SQL.
- Deberá existir una separación clara entre presentación, comunicación, lógica de negocio y persistencia.
- Los endpoints deberán poder ser probados mediante solicitudes HTTP.
- En la Etapa 2, el front-end deberá consumir la API desarrollada durante la Etapa 1.
- Cada integrante deberá demostrar conocimiento suficiente sobre la solución presentada.

El incumplimiento de alguno de los componentes fundamentales podrá afectar la aprobación de la etapa, independientemente del funcionamiento de otras partes del sistema.